

Dublin Bikes — Project Report

UCD School of Computer Science — Software Engineering Project

Kaiwen Yao • Tzuyu Chang • Ziling Huang April 2026

Authors and Contributions — each author contributed an agreed equal share of **33.33%**.

Author	%	Type of contribution
Kaiwen Yao	33.33	Code (<code>flask-app</code> : JWT auth, AI chat SSE, DB schema/migrations, stations/weather/prediction APIs); Report (Process, Architecture & Design); Management (CI/Jenkins, Docker).
Tzuyu Chang	33.33	Code (<code>react-app</code> : map UI, journey planner, chat UI, Tailwind styling); Report (Overview, Requirements, mockups, user stories); Management (sprint reviews/retros).
Ziling Huang	33.33	Code (<code>scraper</code> : JCDcaux & OpenWeather ingest; ML pipeline <code>ml.ipynb</code> , training, model export); Report (Machine Learning, Testing); Management (product backlog, data/model QA).

Screen Recording (6–8 min max). URL: *TODO* — *paste final video URL*. 3–5 min: feature walkthrough with voice, browser URL bar clearly showing the **EC2 address**. ~3 min: each member presents **1 min** on their contributions (Kaiwen → Tzuyu → Ziling).

GitHub Project. Docs/report: <https://github.com/ucdse/docs> · Backend: <https://github.com/ucdse/flask-app> · Frontend: <https://github.com/ucdse/react-app> · Scraper: <https://github.com/ucdse/scraper>

Product & Sprint Backlogs. Product backlog: *TODO* — *URL*. Sprint 1: *TODO*. Sprint 2: *TODO*. Sprint 3: *TODO*. Sprint 4: *TODO*.

Generative AI Chats (one per student, in the docs repo). KaiwenYao_Generative_AI_chats.md · TzuyuChang_Generative_AI_chats.md · ZilingHuang_Generative_AI_chats.md

1. Overview

1.1 Project Objectives and System Architecture

The primary objective of this project is to develop a comprehensive, high-interaction web application that addresses the "last-mile" transportation challenges in Dublin by integrating real-time data, meteorological information, and predictive analytics. The system provides users with live occupancy status for Dublin Bikes stations while leveraging machine learning to assist in travel decision-making under Dublin's unpredictable weather patterns.

Architecturally, the project utilizes a decoupled client-server and cloud architecture, strictly separating front-end user interfaces, back-end web routing, and data acquisition processes.

The work is split across three independent Git repositories, each with a dedicated CI/CD pipeline:

- **flask-app**: The REST API backend managing authentication, station availability, ML predictions, and AI chat logic.
- **react-app**: A TypeScript-based single-page application (SPA) providing a dynamic and responsive user experience.
- **scraper**: A standalone data ingestion engine for JCDecaux and OpenWeather APIs.

The system is containerized via Docker and deployed on AWS EC2, utilizing AWS RDS (MySQL) for centralized, high-performance data persistence, ensuring that long-running ingestion tasks do not block API responses.

1.2 Target Users and Personas

Based on the requirements elicitation process in Sprint 1, we identified three core personas to guide our feature development:

- Eoin (The Daily Student Commuter): A budget-conscious student relying on cycling for his 9:00 AM lectures. His primary pain points are finding empty stations and getting caught in rain.
 - Solution: Integrated weather forecasts and a 24-hour bike availability prediction model.
- Sarah (The Punctual Office Worker): A professional commuting to the Silicon Docks who requires maximum efficiency and route certainty.
 - Solution: The Journey Planner feature, which calculates the optimal "walk-cycle-walk" route to minimize total travel time.
- Matteo (The Weekend Tourist): A visitor unfamiliar with the city layout who needs an intuitive interface to find stations near landmarks.
 - Solution: High-interaction map visualization and a Generative AI Chatbot for natural language assistance.

1.3 Core Functional Features

A. Interactive Map Visualization & Weather Integration

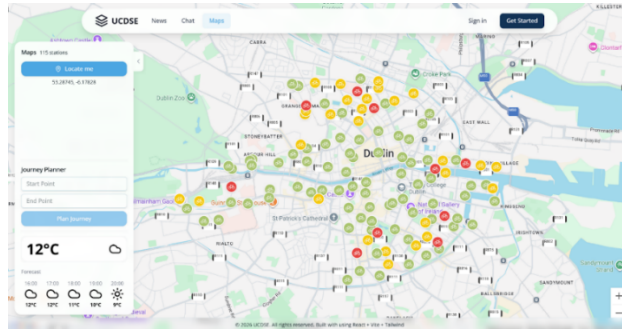


Figure 1.1: Main interactive dashboard with dynamic station status and real-time weather integration.

Built with React and the Google Maps SDK, the interactive dashboard provides a real-time visual representation of the Dublin Bikes network alongside an integrated weather forecasting utility. A dynamic color-coding mechanism (green, yellow, red) enables rapid assessment of station availability, while station click events trigger asynchronous API requests to fetch detailed occupancy history. To assist with travel decisions, the frontend consumes meteorological data—fetched hourly from the OpenWeatherMap API by a background scraper and persisted in AWS RDS—displaying live conditions and upcoming forecasts seamlessly within the same interface.

B. Comprehensive Station Dashboard & Predictive Analytics

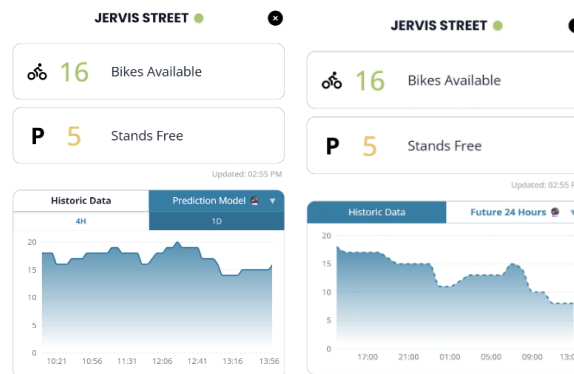


Figure 1.2: ML-driven 24-hour bike availability forecast for a selected station.

Upon clicking a station marker, users are presented with a detailed, interactive pop-up window. This dashboard provides an immediate snapshot of the station's real-time status, clearly displaying the exact number of available bikes and free stands. Beyond live metrics, the interface features a dynamic, dual-tabbed chart component. Users can seamlessly toggle between "Historic Data" to analyze recent occupancy trends and the "Prediction Model" to view a 24-hour machine learning-driven forecast.

1.4 Advanced Optional Features

A. Global Minimum Duration Journey Planner

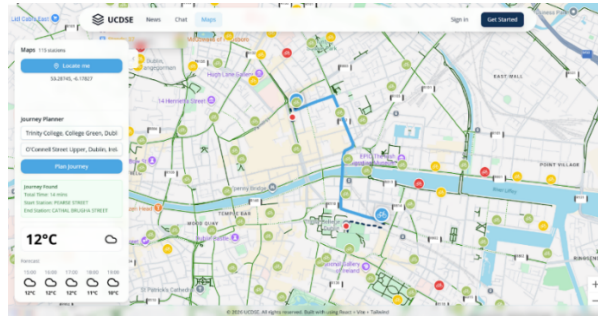


Figure 1.3: Journey Planner demonstrating the globally optimized "walk-cycle-walk" routing.

The Journey Planner utilizes a Global Minimum Duration Algorithm to recommend the most time-efficient route for users. By analyzing real-time station availability alongside multi-modal travel data from Google Maps, the system dynamically identifies the fastest "walk-cycle-walk" combination to minimize the overall commute time. This ensures that users are guided to operative stations with confirmed bike or stand availability, providing a reliable and optimized travel experience.

B. Secure Authentication and Session Management

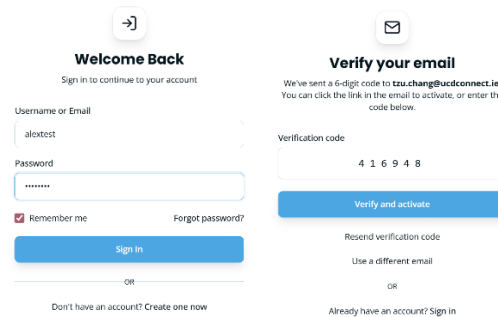


Figure 1.4: Secure JWT-based authentication and account activation interface.

The system features an enterprise-grade security layer utilizing JWT (Access/Refresh Tokens) and 6-digit email verification. Authentication states are isolated in the user and sessions tables of the AWS RDS instance, ensuring secure access to personal chat histories and profile settings.

C. Stateful Generative AI Support

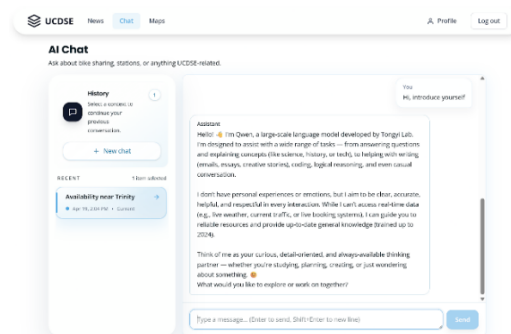


Figure 1.5: LLM-powered assistant with session persistence and natural language processing.

Leveraging the LangChain framework and Aliyun Qwen API, we developed a stateful AI

assistant. The backend utilizes native JSON columns in the message_store table to archive complex chat interactions, allowing the model to maintain context across sessions and automatically classify chat topics.

2. Requirements

2.1 Requirements Elicitation Process

To ensure the Dublin Bikes application delivers meaningful value to its diverse user base, we implemented a structured, user-centric requirements elicitation process during Sprint 1. The goal was to bridge the gap between high-level project objectives and concrete technical specifications through empirical research and stakeholder analysis.

The process followed four distinct phases:

- **Market Research & Competitive Analysis:** We conducted a benchmarking study against existing bike-sharing platforms such as the official TFI Bikes app, Villo! (Brussels), and Santander Cycles (London). This research identified a significant market gap in Dublin: the lack of integrated predictive availability and real-time weather-sensitive routing.
- **Qualitative User Interviews:** To understand local pain points, we conducted structured interviews with three primary stakeholders. These sessions focused on daily commuting habits, the impact of Dublin's unpredictable weather, and the frustrations associated with "station-full" or "station-empty" scenarios.
- **Persona Synthesis:** Insights from the interviews were synthesized into three distinct personas: Eoin (the budget-conscious student), Sarah (the punctual professional), and Matteo (the weekend tourist). These personas served as the "living documentation" that guided every design decision.
- **User Story Mapping:** Finally, we translated user needs into formal User Stories and Acceptance Criteria (AC). This provided us with a clear "Definition of Done" for each feature and formed the basis of our automated testing suite.

2.2 App Mockups and Design Rationale

The following mockups represent the most critical interfaces of the application, designed to address the specific needs identified during the elicitation process.

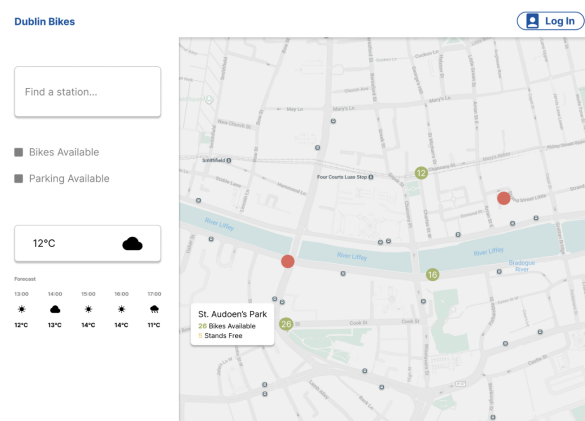


Figure 2.1: Primary dashboard featuring the Google Maps SDK integration and weather widget.

Design Focus: This interface establishes foundational situational awareness for the user. By utilizing a clean, full-screen map layout paired with interactive hover states. Additionally, the integrated sidebar features a streamlined search function for rapid spatial navigation, alongside a real-time weather widget that facilitates immediate "go/no-go" travel decisions.

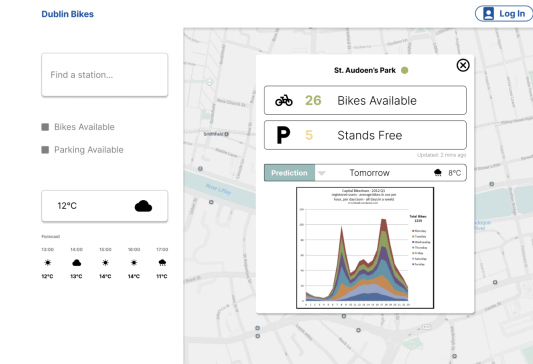


Figure 2.2: Detailed station view showing the toggle between Historical Trends and ML Predictions.

Design Focus: To alleviate Sarah's anxiety regarding morning meetings, this dashboard provides predictive certainty. By clicking a station, users can view a 24-hour forecast powered by our Random Forest model, allowing for preemptive planning of commutes based on projected availability rather than just live snapshots.

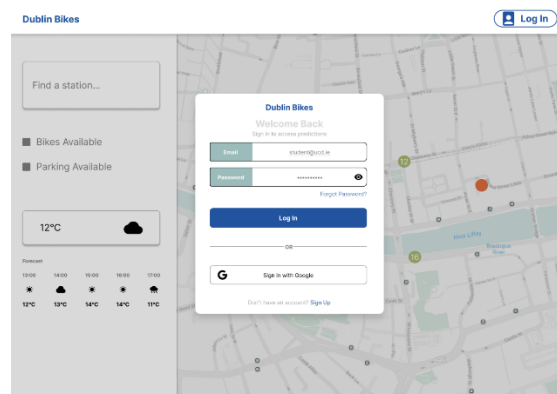


Figure 2.3: User login interface.

Design Focus: This interface provides a secure and straightforward user authentication process. By facilitating simple registration and login workflows, it ensures that users can safely manage their personal accounts and access customized features, such as their saved AI chat history.

2.3 User Stories and Acceptance Criteria

The following table outlines the core functional requirements that drove the development of the Dublin Bikes ML App.

User Story ID	User Story Description	Acceptance Criteria (AC)

US-01	As a student running late (Eoin), I want to see real-time bike counts on the map, so that I don't waste time walking to an empty station.	1. Markers must update dynamically via JCDecaux API. 2. Clicking a marker must display the exact count of bikes/stands.
US-02	As a commuter planning ahead (Sarah), I want to view predicted availability for the next 24 hours, so that I can plan my morning journey.	1. The "Prediction Model" tab must render an ML-driven chart. 2. Predicted values must be mathematically clamped between 0 and station capacity.
US-03	As a time-sensitive professional (Sarah), I want to use a journey planner to find the fastest route, so that I arrive at my meetings efficiently.	1. System must calculate the optimal Walk-Cycle-Walk combination. 2. The map must render a polyline route with an associated ETA.
US-04	As a tourist (Matteo), I want to ask questions using natural language via an AI assistant, so that I can understand rental rules easily.	1. The GenAI Chatbot must process queries using LangChain/LLM. 2. Conversation history must be persisted for authenticated users.
US-05	As a frequent user, I want to register and log in securely, so that I can access personalized features like saved routes and AI chat.	1. Registration must require a 6-digit email verification code. 2. The system must issue and validate secure JWT tokens.

2.4 Supplementary Material

To maintain a concise report, the extensive raw data generated during the requirements phase has been archived. This includes:

- Full Interview Transcripts: Comprehensive logs from sessions with Sarah, Matteo, and Eoin.
- Detailed Persona Profiles: In-depth analysis of user demographics, goals, and frustrations.
- Competitive Analysis Matrix: A technical feature comparison across major European bike-sharing systems.

Reference Note: All aforementioned comprehensive documentation related to our elicitation process can be accessed in our GitHub repository at:

<https://github.com/ucdse/docs/tree/main/requirements>

3 Architecture and Design

3.1 Overall Architecture

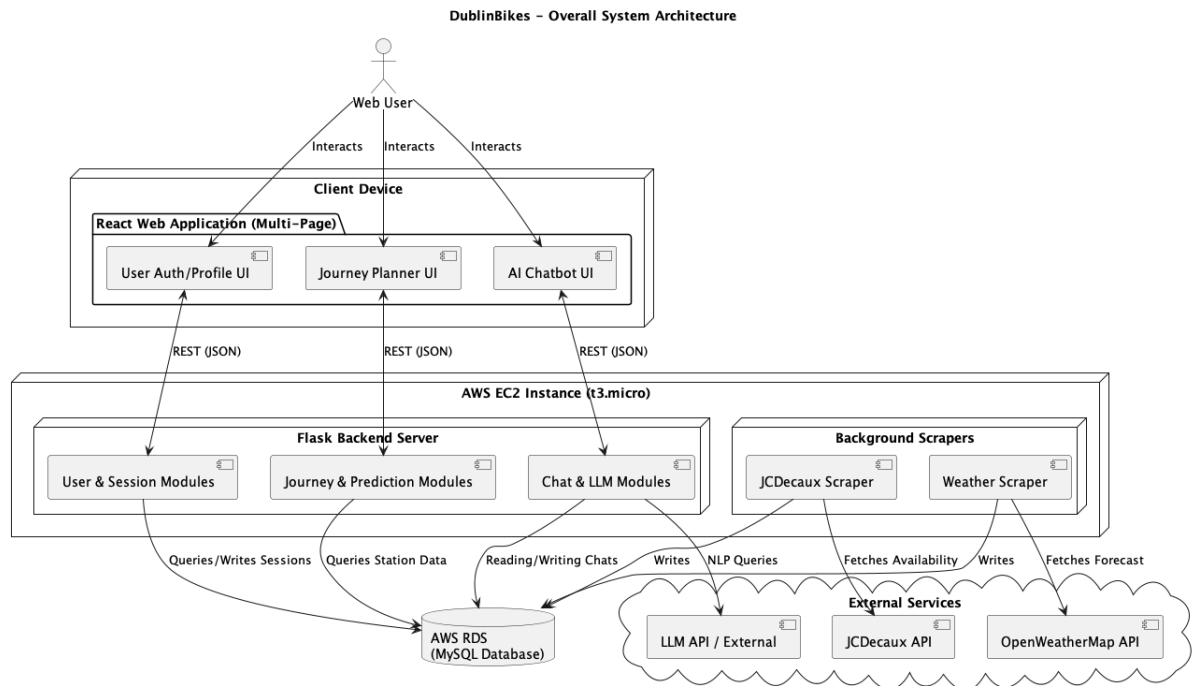


Figure 1 Overall System Architecture

Our project utilizes a decoupled client-server and cloud architecture, separating the front-end user interfaces, back-end web routing, and the data acquisition processes.

The architecture (Figure 1) illustrates a scalable Web application split into three robust functions: user authentication, journey planning, and an AI chatbot.

Client Device: A React-based multi-page application (MPA) utilising distinct route pages (/login, /maps, /chat). It communicates to the backend asynchronously via JSON REST APIs seamlessly across the layout.

Server Environment (AWS EC2): A scalable AWS EC2 micro instance hosts our core Python layer.

Flask Backend Server: Acts as the traffic controller, routing API requests, formatting data, and calling our predictive models.

Background Scrapers: Standalone scripts continuously gather live data from the JCDecaux API and OpenWeatherMap API, pushing raw data into our database independent of the Web Server.

Database Layer (AWS RDS): A managed, centralised AWS RDS MySQL instance that securely maintains static station references, dynamic availability histories, and weather data independent of the EC2 lifecycle.

This separation of concerns guarantees that long-running data-fetching tasks don't block API responses, and ensures that the client remains highly responsive.

3.2 Class Diagram

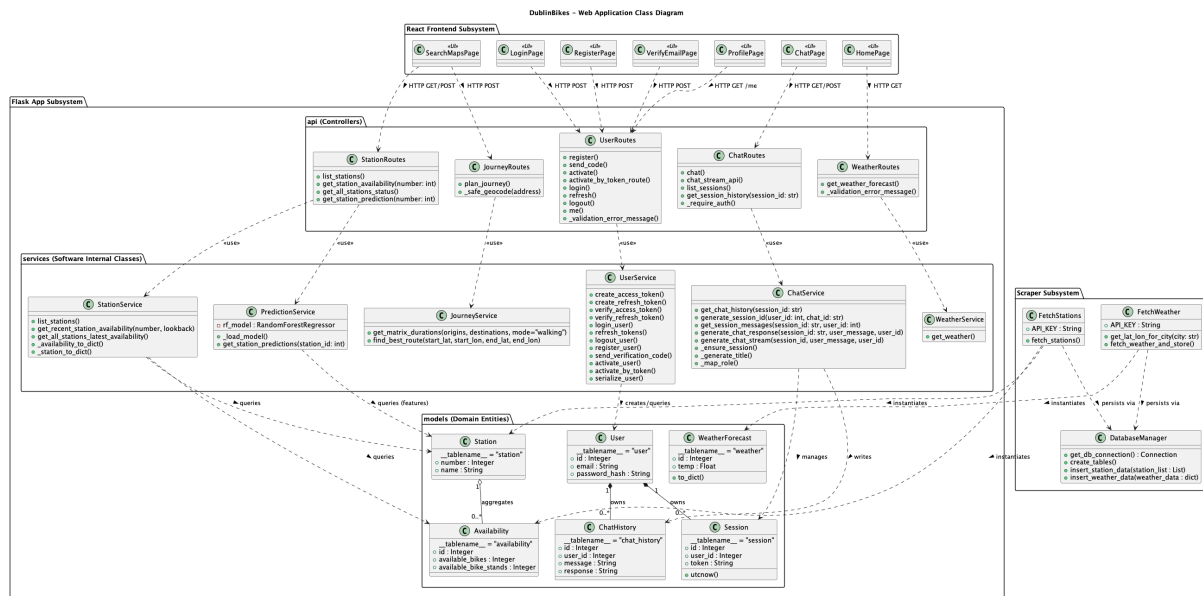


Figure 2 Web Application Class Diagram

The detailed Class Diagram (Figure 2) abstracts our underlying code into proper object-oriented software internal classes and domain entities.

Scraper Subsystem: Exhibits a collection mapping to core ingestion processes. It uses “FetchStations” and “FetchWeather” logic which delegate raw formatting to an internal “DatabaseManager” to persist data instances.

Flask App Subsystem: Illustrates our service-oriented architecture covering the three main functions.

Models (Domain Entities): Database-bound objects mapping our RDS tables, including our core domain (“Station”, “Availability”, “WeatherForecast”), alongside user and AI chat state logic (“User”, “Session”, “ChatHistory”).

Services (/services): Business logic implementations explicitly separated: “UserService” (authentication), “JourneyService” (route predictions), and “ChatService” (message processing).

Routes (/api): The Flask routing layer intercepts specific domain requests (“UserRoutes”, “JourneyRoutes”, “ChatRoutes”) and forwards the payload dictionaries into the respective services without holding heavy calculation logic itself.

React Frontend Subsystem: Outlines our front-end functional UI components (“LoginPage”, “SearchMapsPage”, “ChatPage”) and how they trigger specific “HTTP POST/GET” requests to their corresponding API branches in the Flask-App schema.

3.3 Sequence Diagrams of Core Use Cases

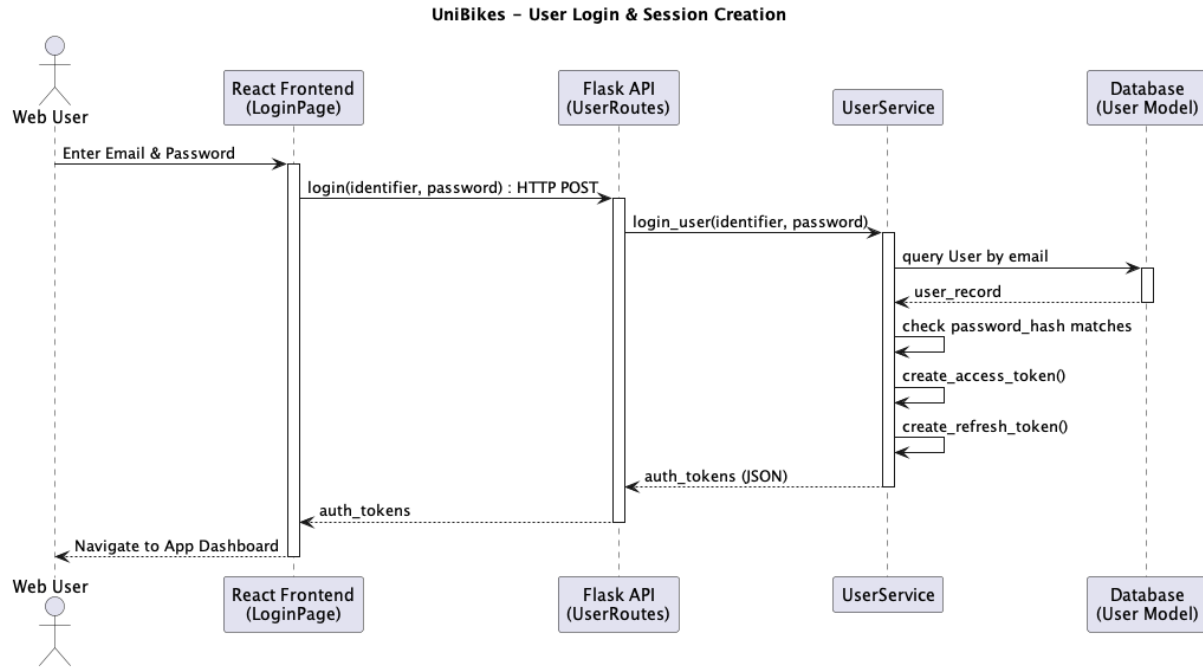


Figure 3 User Login and Session Creation

User Login (Figure 3): Details the flow from the React “LoginPage” to the generation of JWT auth keys in the “UserService”, concluding with the issuance of stateless JWT access and refresh tokens back to the client interface.

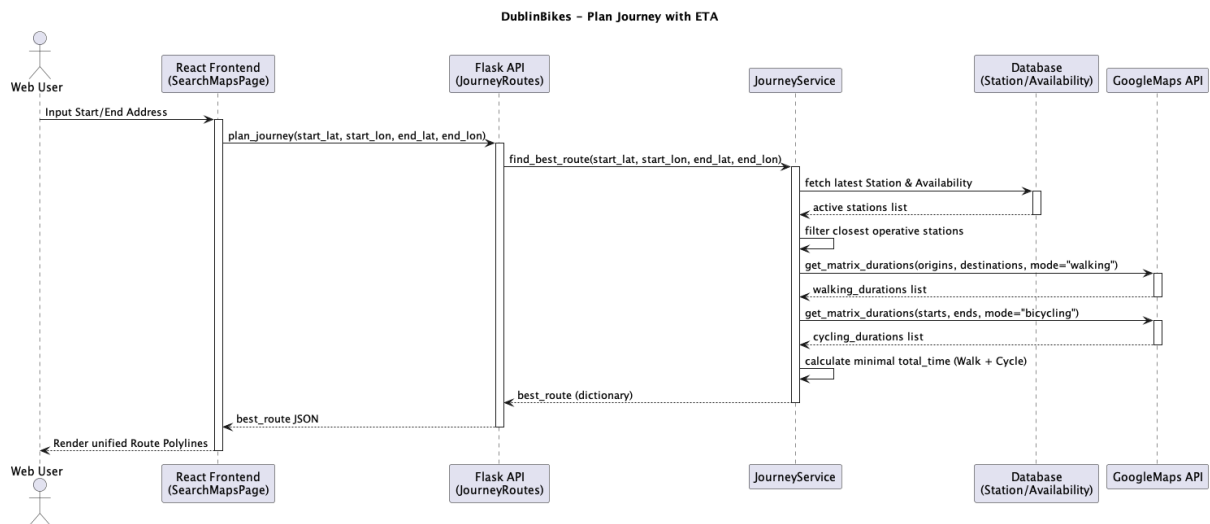


Figure 4 Journey Planner

Journey Planner (Figure 4): Initiated from “SearchMapsPage”, the “JourneyService” utilises matrix computations directly against the GoogleMaps API (“get_matrix_durations()”), combining optimal cycling and walking computations geographically.

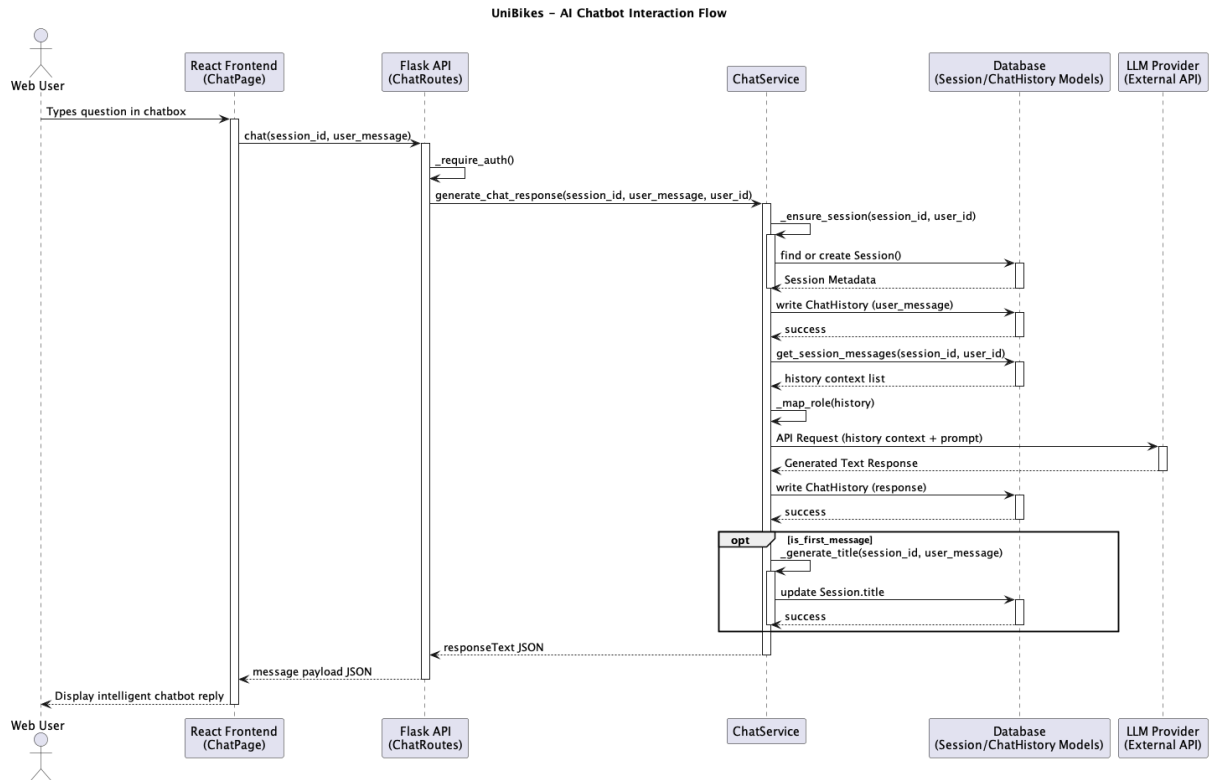


Figure 5 AI Chatbot

AI Chatbot (Figure 5): The message passes structural auth checks (“_require_auth()”) and persists domain state via “ensure_session()”, referencing the “Session” metadata table. The “ChatService” fetches interaction context (“ChatHistory”) prior to transmitting prompts to the external LLM provider, while dynamically utilising the LLM (“_generate_title()”) to auto-classify a chat topic title on initial interaction.

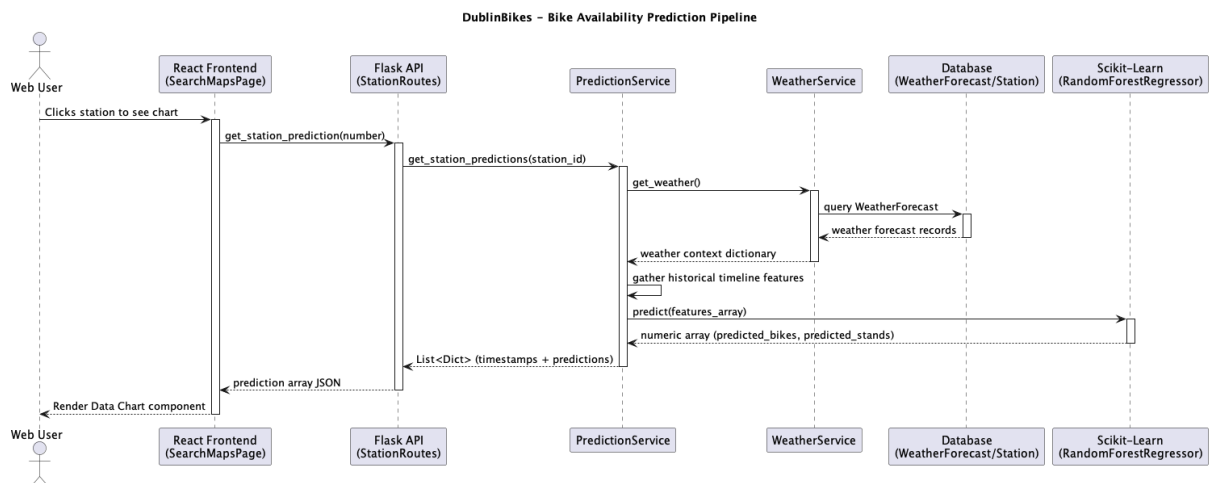


Figure 6 Bike Availability Prediction

Bike Availability Prediction (Figure 6): The request flows from “StationRoutes” down to the “PredictionService”, appending contextual features mapped from “WeatherService” against the local “Database” cache before engaging the predictive memory-bound “RandomForestRegressor”. To

address typical initial cold-start latencies inherent to ML modeling, the “PredictionModel” referenced in the prediction sequence is pre-loaded directly into Flask's memory upon initialization.

3.4 Database Design and Design Choices

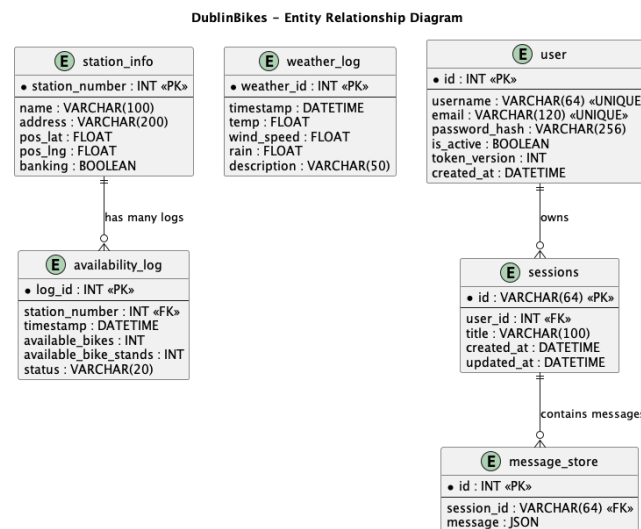


Figure 7 Entity Relationship

Managed AWS RDS (Relational Cloud SQL): We opted for a cloud-managed Relational Database (AWS RDS for MySQL) rather than a local SQLite file or NoSQL. Shared bike systems and meteorological data are highly structured datasets. Given that the Machine Learning modeling pipeline requires performing time-series transformations and heavy “JOIN” operations across station IDs and timestamps, a robust cloud SQL environment was naturally the correct fit.

Data normalization: Static factors like a station's latitude, longitude, and internal naming (“station_info”) remain constant. Instead of redundantly saving these details with every availability check, they are separated. The “availability_log” records highly volatile factors (every 5-10 minutes) leveraging Foreign Keys mapping back to the static tables, reducing storage footprint and read redundancy.

Indexing: Because querying time-aligned weather logs and bike availability histories spans millions of rows, we introduced specific database indexing on the “timestamp” and “station_number” columns. This design drastically diminished query delays when compiling features to refresh our predictive Random Forest classifiers.

Stateful LLM persistence & auth isolation: To securely expand the application beyond public bike metrics, we isolated authentication and Chatbot states via the “user”, “sessions”, and “message_store” tables. The “user” records enforce robust identity and JWT validations, mapped via foreign key explicitly to Chatbot metadata (“sessions”). To accommodate the vast, unpredictable text structure characteristic of Large Language Models via LangChain, the “message_store” table purposefully archives deep ongoing chat interactions leveraging native JSON columns.

3.5 Additional Material

All original UML files are securely tracked in our GitHub repository
https://github.com/ucdsc/docs/tree/main/UML_files.

4. Machine Learning Model

This chapter describes the machine learning component of the Dublin Bikes project. The model predicts the number of available bikes at any station for each weather forecast entry cached in the database (up to 48 hours ahead). Training code and exported artefacts (bike_availability_model.pkl, model_features.pkl) are stored under flask-app/machine_learning/.

4.1 Problem Definition

The goal is to help users decide when and where to pick up a bike by forecasting future availability. The problem is framed as a supervised regression task:

- Target variable: num_bikes_available — the number of bikes that can be rented from a station at a specific point in time.
- Inputs: a vector of temporal, spatial, and meteorological features that are known in advance or can be observed at prediction time.

Predictions are generated in bulk for every forecast entry stored in the weather_forecast table whose forecast_time is at or after the current hour. Results are exposed to the frontend via the GET /api/stations/<number>/prediction endpoint, where <number> is the JCDecaux station number used as the primary key in the station table.

4.2 Dataset and Feature Engineering

4.2.1 Raw Data

The training set is final_merged_data.csv (located in flask-app/machine_learning/), containing 298,946 rows and 78 columns. It was built by joining historical station-availability snapshots with historical weather readings provided by Met Éireann. Each row represents the state of one specific station at one specific hour, augmented with the weather conditions recorded during that hour. An initial exploratory analysis confirmed that the dataset contains no missing values, so no imputation step was required.

4.2.2 Feature Selection

The following column groups were removed:

Removed Columns	Reason
*_quality_indicator	Metadata flags with no meteorological signal
*_std_deviation	Redundant once min/max values are retained
Soil/grass/earth temperature	Low correlation with urban bike usage
Text columns (name, address, etc.)	Location encoded by station_id, lat, lon
Near-constant booleans (is_installed, etc.)	Almost 100% True; no discriminative power

4.2.3 Engineered Features

To capture periodic and meteorological patterns more explicitly, five derived features were created from the reduced column set:

- day_of_week — integer 0 (Monday) to 6 (Sunday), extracted from the date column.
- is_weekend — binary indicator (1 if Saturday or Sunday, 0 otherwise).

- `avg_temperature` — arithmetic mean of `max_air_temperature_celsius` and `min_air_temperature_celsius`.
- `avg_humidity` — arithmetic mean of `max_relative_humidity_percent` and `min_relative_humidity_percent`.
- `avg_pressure` — arithmetic mean of `max_barometric_pressure_hpa` and `min_barometric_pressure_hpa`.

The averaged values were preferred over raw min/max pairs to reduce collinearity while preserving the same physical signal. A correlation check also revealed that month and year had zero variance across the dataset (which covers December 2024 only); both were dropped as they cannot generalise to other months.

4.2.4 Final Feature Set (11 features)

The model learns from the following 11 input features, in the exact order required by the serialised pipeline:

- `station_id` — the JCDecaux station number (primary key in the station table).
- `capacity` — total number of docking stands at the station.
- `lat / lon` — station latitude and longitude.
- `hour` — hour of day (0–23); `day` — day of month (1–31).
- `day_of_week` — weekday index (0–6, Monday = 0); `is_weekend` — weekend flag (0/1).
- `avg_temperature` (°C), `avg_humidity` (%), `avg_pressure` (hPa).

Note: at inference time, the `avg_*` features are populated from hourly OpenWeatherMap point forecasts rather than daily min/max aggregates. The column names are kept identical so that the ordering matches `model_features.pkl`, but the underlying distributions differ slightly from those seen during training.

4.3 Training and Evaluation

The cleaned dataset was split using scikit-learn's `train_test_split` with a 70/30 ratio and `random_state=42` for reproducibility, yielding 209,262 training rows and 89,684 test rows. Four regression algorithms were trained and evaluated under identical conditions:

- Linear Regression — a simple baseline to establish a lower bound on performance.
- Decision Tree Regressor — captures non-linear interactions without requiring feature scaling.
- Random Forest Regressor — an ensemble of decision trees, configured with `n_estimators=100`, `max_depth=15`, and `min_samples_leaf=10` for modest regularisation.
- Gradient Boosting Regressor — builds trees sequentially to correct residuals from previous iterations.

Performance was measured with three standard metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).

Model	MAE	RMSE	R^2
Linear Regression	7.829	9.366	0.075
Decision Tree	0.970	2.376	0.941
Random Forest (selected)	2.325	3.393	0.879
Gradient Boosting	6.120	7.537	0.401

Linear Regression performed poorly ($R^2 \approx 0.08$), confirming that bike availability is not linearly separable. The Decision Tree achieved the lowest MAE (0.970) and highest R^2 (0.941), but its perfect fit on training data raised concerns about overfitting. The Random Forest Regressor was therefore selected as the final model: it sacrifices a small amount of accuracy compared to the Decision Tree while offering meaningfully better generalisation and resistance to noise.

4.4 Production Integration

The trained Random Forest and the ordered feature list were serialised with pickle as `bike_availability_model.pkl` and `model_features.pkl` respectively. At inference time, the Flask service constructs a pandas DataFrame with the 11 required features, calls `model.predict()`, and clamps each output to the range `[0, station.bike_stands]` to prevent physically impossible values.

The most important features, in descending order of contribution, were: `lat/lon` (geographic position), `day` and `hour` (calendar specificity), `station_id`, `avg_pressure`, `capacity`, `day_of_week`, `avg_temperature`, `avg_humidity`, and `is_weekend`.

4.5 Reflection and Future Work

Strengths

- The pipeline is straightforward and fast to retrain — no missing-value imputation is needed and the feature set is compact.
- Predictions are directly actionable: the output is a bike count, not an abstract score, making it easy to display to end users.
- The REST endpoint decouples the model from the frontend, allowing both to be updated independently.

Limitations

- Temporal scope: training data covers only December 2024. Seasonal variation (summer vs. winter commuting patterns) is not captured, which may reduce accuracy in other months.
- Weather-source mismatch: training uses Met Éireann daily min/max aggregates while inference uses OpenWeatherMap hourly point forecasts, introducing a systematic distributional shift in the `avg_*` features.
- The Decision Tree's superior in-sample metrics (MAE 0.970, R^2 0.941) compared to the Random Forest suggest potential for further gains if overfitting can be controlled through deeper regularisation or cross-validation.

Future Improvements

- Expand training history to cover multiple months and seasons for better generalisation.
- Add lag features (e.g., bike count at $t-1$ hour) to exploit temporal autocorrelation.
- Perform systematic hyperparameter optimisation (e.g., grid search or Bayesian optimisation) on the Random Forest.
- Introduce prediction confidence intervals to communicate uncertainty to users.

5. Testing and Quality Assurance

5.1 Strategy and Coverage Results

To ensure the reliability, security, and performance of the Dublin Bikes backend architecture, we implemented a robust automated testing strategy. Built using Pytest (v9.0.3) and pytest-cov, our approach encompassed both isolated Unit Tests and end-to-end Integration Tests for the Flask API layer.

To ensure hermetic testing environments, we configured confest.py to route all test operations to an isolated in-memory SQLite database. This guaranteed fast execution and prevented any pollution of the production MySQL instance. The suite executed 355 test cases in just 12.5 seconds, achieving an exceptional 97% Statement Coverage across 1,272 lines of code.

Coverage Breakdown:

- API Routing & Data Contracts: 95% - 100%
- Journey, ML Prediction & Weather Services: 100%
- User Authentication Service: 96%
- AI Chat Service: 92%

5.2 Key Testing Scenarios and Methodologies

Achieving deterministic results required addressing external dependencies and complex boundary conditions:

- External API Mocking: We extensively utilized unittest.mock.MagicMock to intercept third-party network calls at the module level. We mocked the Google Maps client to test our "walk-cycle-walk" routing offline, stubbed OpenWeather to verify temporal filtering logic, and intercepted LLM interactions. This guaranteed 100% test determinism and prevented API quota exhaustion.
- ML Boundary Clamping: Using robust test fixtures (make_weather_forecast, make_station), we wrote explicit unit tests to guarantee that the Random Forest model's probabilistic outputs are always mathematically clamped between zero and a station's maximum dock capacity.
- Security Validation: To secure our JWT-based infrastructure, the suite simulated various unauthorized access attempts using custom auth_headers fixtures. We strictly tested token expirations, secret key mismatches, and time-to-live (TTL) boundaries to ensure correct HTTP 401 responses.

6. Process

6.1 Project Organisation and Tools

The project is split across three Git repositories, each with its own Jenkins CI/CD pipeline deploying to AWS EC2 via Docker:

Repository	Role
flask-app	REST API (Flask/Gunicorn), JWT auth, ML predictions, journey planning, AI chat. Owns the MySQL schema via Flask-Migrate.
react-app	Single-page frontend (React 19, TypeScript, Vite 7, Tailwind CSS 4). Served by nginx in production.
scraper	Long-running Python process: polls JCDecaux stations and OpenWeatherMap forecasts, writing to the shared database.

Tooling: Git feature-branch workflow • Jenkins (Kubernetes agent) • Docker • AWS EC2 • pytest ($\approx 97\%$ coverage) • ESLint / TypeScript • Discord (team communication).

Integration contract: Both flask-app and scraper share one DATABASE_URL. Flask-Migrate migrations must run first. In development, Vite proxies /api to Flask on port 5000; in production a reverse proxy handles the same routing.

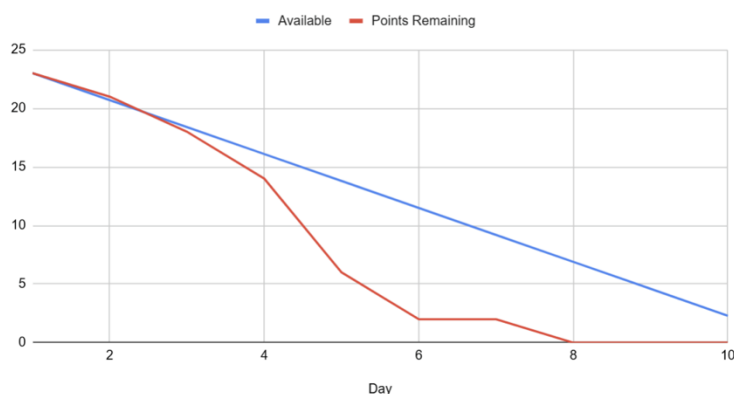
6.2 Sprint 1 — Requirements Engineering & Data Collection

Implemented Features and Design Decisions

The team conducted user interviews to validate three personas (Student, Professional, Tourist), produced Figma UI mockups, and defined user stories. On the technical side, a MySQL database schema was established and scraping scripts for the JCDecaux and OpenWeatherMap APIs were written and deployed. A 5-minute scrape interval was chosen as the design default to balance data freshness against API rate limits. The scraper ran stably throughout the sprint, consistently populating the database and providing a reliable foundation for subsequent development.

Burndown Chart

Available and Points Remaining



The Actual Remaining Effort line crossed below the Ideal Trend line on Day 4 and reached zero two days before the deadline, indicating the team worked ahead of schedule. The surplus time was used for backlog refinement and feasibility research on the Smart Journey Planner feature.

Sprint Review

- Presented Figma mockups aligned to persona pain-points (tourist navigation, weather prediction, map interactivity).
- Demonstrated live MySQL tables populated by the running scrapers.
- Opened a scope discussion on the Journey Planner feature; decision deferred to Sprint 2 planning.

Sprint Retrospective

What went well: Discord channel established for daily updates. User interviews successfully validated all three personas, directly shaping the decision to prioritise map interactivity. Scraping scripts delivered ahead of schedule and ran stably.

What could be improved: Some members were unfamiliar with API requests and local MySQL configuration, causing early delays. Peer support will be increased in future sprints to resolve technical blockers faster.

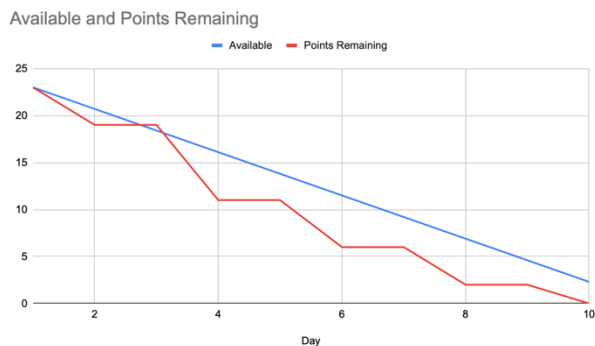
For next sprint: Set up the Flask application and connect it to the database; get scraped data displaying on a live map; finalise the Journey Planner feasibility study.

6.3 Sprint 2 — Backend Integration & Deployment Pipeline

Implemented Features and Design Decisions

The Flask application was built out with Weather and Station Read APIs returning properly formatted JSON from the shared MySQL database. JWT-based user authentication was integrated with secure password hashing. A Jenkins CI/CD pipeline was configured with a Kubernetes agent, automating Docker builds and EC2 deployments on every push to main. A Git feature-branch workflow was adopted team-wide. Crucially, the Journey Planner backend endpoint was delivered, closing the open scope question from Sprint 1, though it required significant refactoring to handle edge cases and correct route-calculation logic.

Burndown Chart



The chart shows a staircase pattern reflecting the mandated two-day update cadence. Despite this, the Actual Remaining Effort line dropped below the Ideal Trend by Day 4 and stayed beneath it for the rest of the sprint, confirming strong team velocity. All goals were completed by Day 10.

Sprint Review

- Demonstrated Weather and Station APIs returning live JSON data.
- Showcased JWT login flow and hashed password storage.
- Live CI/CD demo: a code push triggered a Jenkins build and automatic EC2 deployment.
- Confirmed Journey Planner inclusion and basic backend functionality.

Sprint Retrospective

What went well: The Git feature-branch workflow and Jenkins pipeline accelerated testing and removed manual deployment effort. Resolving the scope question early enabled delivery of the complex Journey Planner and secure login features.

What could be improved: API endpoints lacked documentation at the outset, forcing frontend developers to read backend source code to find correct routes. The Journey Planner also required substantial refactoring for edge-case handling.

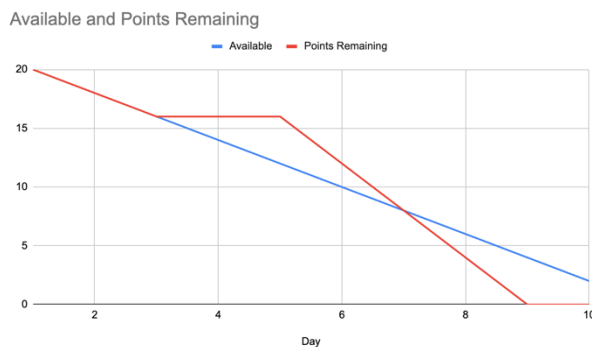
For next sprint: Build React UIs for the Journey Planner and Login; migrate the database to Amazon RDS; produce API documentation to streamline frontend–backend integration.

6.4 Sprint 3 — Additional Features & Frontend Implementation

Implemented Features and Design Decisions

Three AI chat features were implemented using the Langchain framework and the Aliyun Qwen API, with chat history persisted in new database tables and displayed on login. The React frontend was updated to show available bike and stand counts on station hover, and data is now pre-fetched on page load to eliminate display latency. The database was migrated from a local instance to Amazon RDS to overcome EC2 memory constraints and improve query performance. The CI/CD pipeline was verified end-to-end with Docker containerisation confirming a stable, reproducible deployment process.

Burndown Chart



The burndown chart was not explicitly described in the sprint records; the team confirmed all sprint goals were completed within the sprint window.

Sprint Review

- Demonstrated AI chat with history storage and LLM memory via new database tables.
- Confirmed Flask backend completeness through Postman testing.
- Showed hover-based bike/stand data on the map with zero page-load latency.
- Verified Jenkins CI/CD and Docker containerisation for both development and production.

Sprint Retrospective

What went well: Core features are working. Frontend and backend integration was smooth and error-free. Development and production environments are correctly configured, and the code-review workflow is operating well.

What could be improved: UI/UX design and colour scheme need further refinement. API keys risk exposure in the browser console, a security concern for production. AI chat output can be interrupted if the user navigates away mid-stream.

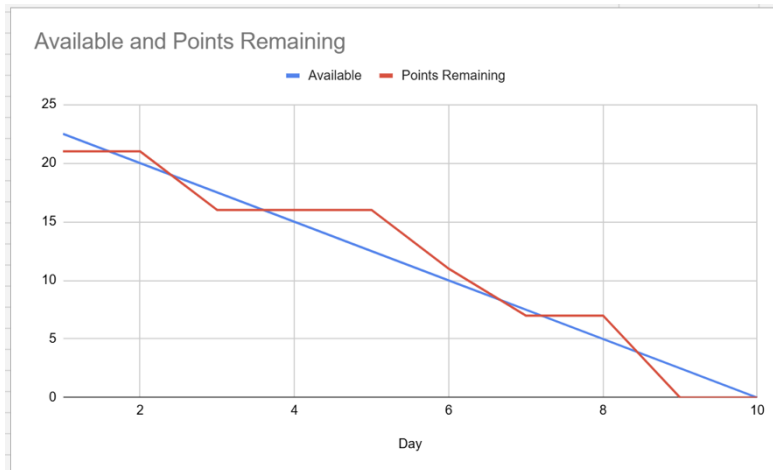
For next sprint: Improve UI/UX and colour scheme; fix AI chat response persistence; implement ML-based availability predictions; add unit, integration, and end-to-end tests.

6.5 Sprint 4 — Machine Learning Integration & System Finalisation

Implemented Features and Design Decisions

A Random Forest model was trained on historical station and weather data, serialised as a .pkl file, and integrated into the Flask backend via a prediction service that preloads the model at startup. The React frontend was updated to fetch and render predictions dynamically, with a brief cold-start latency on the first request acknowledged and documented. UI styling was refined for consistency across all pages. Backend test coverage reached 97% with pytest, and the CI/CD pipeline was updated to download ML artefacts from Hugging Face Hub during the build stage.

Burndown Chart



Points Remaining stayed slightly above the Ideal Trend line during Days 3–5 as the team focused on ML fine-tuning and API formatting issues. A steep drop on Day 7 restored velocity, and all tasks were completed on Day 9, one day before the deadline.

Sprint Review

- Demonstrated the fully integrated web application: React frontend → Flask backend → ML predictions rendered live.
- Explained cold-start latency on first prediction request and confirmed fast performance on subsequent requests.
- Highlighted 97% unit test coverage as evidence of system reliability.
- Agreed final phase will focus on code refactoring, UI/UX polish, and report compilation.

Sprint Retrospective

What went well: The full ML pipeline — data cleaning, feature selection, model training, export, and Flask integration — was delivered successfully. React UI components were implemented and styled consistently. 97% test coverage was achieved, confirming system robustness.

What could be improved: Algorithm comparison and hyperparameter tuning took longer than planned. Minor API formatting issues during frontend–backend integration required extra debugging effort. Cold-start prediction latency was identified as a known limitation.

Final conclusion: All core development tasks are complete. Data scraping, the Flask backend, the React frontend, and ML predictions have been successfully integrated into a fully functional web application.